

04_m7_m9

M7 - Minimal Loop: ONE Grounded LLM Brief [Wk2-v1]

This is the v1 centerpiece and the **only** place you deliberately spend the LLM budget. Per LEAN_SCOPE, M7 is NOT the 7-agent FSM (Monitor->Decompose->Diagnose->Critic->Prescribe->Narrator). The lean M7 is a **single grounded LLM call**: a deterministic Python step computes the decomposition + significance + dollar revenue-at-risk (the M5/M6b code you already built and tested), hands those numbers to Claude as a structured `Finding`, and Claude **narrates only** - it writes the exec Decision Brief and one recommended next test. The anti-hallucination doctrine here is different from M0-M6: there is no dbt compile or BigQuery dry-run to catch a wrong number, because the LLM is producing prose, not SQL. The single defense is **faithfulness**: every figure in the brief must trace verbatim to a tool output, and the prompt must forbid the model inventing, rounding, or recomputing any number. You build/test the deterministic layers first (they cost no LLM tokens), then add the brief on top.

Context to load

- `/clear` first (D8). Open a fresh session for M7 - no bleed from the dbt/MCP milestones.
- Auto-loaded: `CLAUDE.md` (repo root) - pins G1-G5, the canonical funnel, `revenue_at_risk = (conv_t0-conv_t1)*sessions_t1*aov`, and the faithfulness rule "Numbers in the brief are tool outputs verbatim" (G2).
- Source of truth to paste/attach: `@docs/AGENT_ARCHITECTURE.md` **sec 6.7 (Narrator)** - specifically the line *"every numeric claim must trace to a tool-output hash; no number the Narrator invented"* and the explicit note *"at M7 the Narrator only calls `render_brief`"*; **sec 4.3** for the `Finding` JSON shape that becomes the brief's input; **sec 13 step 2** for the L1 exit gate (one anomaly -> brief in <5 min, 0 hallucinated columns).
- `@docs/LEAN_SCOPE.md` lines 76-80 - the v1 "one grounded LLM brief" definition and the stretch self-critique pass.
- `@docs/MCP_ARCHITECTURE.md` sec on `report.render_brief` (returns `{brief_md, brief_html}`, raises `EmptyFindings` on `[]`) - the only LLM-facing tool at M7.

Files to provide

- `@helios/diagnose.py` (from the MVP, Wk1) - the deterministic pipeline that finds the largest WoW movement in `session_conversion_rate`, decomposes it by `device_category` / `channel_group`, runs the two-proportion significance test, and

computes the dollar figure. This produces the `Finding`; M7 only adds a narration step after it.

- `@helios/stats/decompose.py` and the significance function - so Claude sees the exact keys it will receive (`mix_effect`, `rate_effect`, `interaction`, `dominant`, `p_value`, `ci_low`, `ci_high`, `revenue_at_risk_usd`, `basis`). It must narrate these, not regenerate them.
- `@models/semantic/semantic_layer.yaml` - the ONLY source of metric/dimension names the brief may use (D7).
- `@helios/mcp/<your-one-server>.py` (semantic-mcp OR stats-mcp from M6b) and `@mcp_servers.yaml` - the governed tool the deterministic layer calls.
- A real recorded `Finding` JSON (one run's output) as the test fixture, so the brief test needs no live LLM/BigQuery.

Prompts to use

Build the deterministic harness first (no LLM tokens spent), test-first:

```
[ROLE] You are implementing Helios milestone M7 (LEAN, Wk2-v1), file helios/brief.py.
[SOURCE OF TRUTH] Copy/adapt the Narrator contract in @docs/AGENT_ARCHITECTURE.md sec 6.7 and the Fi
[UPSTREAM] The Finding comes from @helios/diagnose.py (already built/tested). Reference only these k
[TASK] Write helios/brief.py: a function build_brief(finding: dict) → str that calls the LLM exactl
[CONSTRAINTS] The LLM may NOT invent, recompute, round, or restate any number not present in the Fin
[TEST-FIRST] Write helios/tests/test_brief_faithfulness.py FIRST: load the recorded Finding fixture,
[OUTPUT] Cite the doc section (AGENT_ARCHITECTURE 6.7 / 4.3) used for the brief structure and the fa
```

The single grounded LLM-call prompt embedded inside `build_brief` (this is the product's runtime prompt - keep it terse and number-locked):

```
You are the Helios Narrator. Compose a one-page executive Decision Brief from the Finding JSON below

HARD RULES:
- Every number you write MUST be copied verbatim from the Finding. Do NOT compute, re-derive, round,
- You did not run any analysis. The decomposition, significance test, and dollar figure are already
- Use only the metric and dimension names exactly as they appear in the Finding.
- State the mix-vs-rate verdict using finding.dominant: if "rate", the change is real in-segment beh
- One recommended next test only, as a single sentence; do not design or size an experiment.

Sections: Situation | What moved (metric, slice, t0→t1, delta) | Mix vs rate verdict | Revenue at r

FINDING:
<finding_json>
```

(Stretch, if a day is free - the self-critique nod from LEAN_SCOPE) - a SECOND, cheap LLM call that audits the first against the numbers:

```
[ROLE] M7 stretch: a single self-critique pass in helios/brief.py (build_brief(finding, critique=True)
[TASK] After composing the brief, make ONE more LLM call that receives the brief AND the Finding JSON
[CONSTRAINTS] The critic call may ONLY compare strings to the Finding; it does not fetch data or compute
[TEST-FIRST] Add a test that feeds a deliberately corrupted brief (a hand-edited dollar figure) and
[OUTPUT] Cite LEAN_SCOPE lines 79-80 (the self-critique stretch) and note this is NOT M9's Critic.
```

Preventing hallucinations (specialized - M7 has no compiler to catch a bad number)

- **The numbers come from tools, the LLM only narrates.** Build and test `helios/diagnose.py` + `decompose.py` + the significance test (deterministic, golden-value tested at M5) BEFORE writing `brief.py`. The LLM never sees raw rows and never computes - it receives a finished `Finding` (D2/D3 applied to a JSON payload instead of upstream SQL).
- **Forbid invention explicitly (G2 specialized).** The runtime prompt's HARD RULES forbid computing, re-deriving, rounding, or estimating any figure - the brief may only echo what is in the `Finding`. This is the M7 form of D5: "if a number is not in the attached `Finding`, OMIT it - do not guess."
- **Canonical names only (D7/G5).** The brief may name only metrics/dimensions present in `semantic_layer.yaml`; the test greps the brief for any metric-like token not in the registry and fails on a hit. No raw SQL is in scope at all (D2): the brief step calls zero query tools.
- **Let the test be the machine check (D6 analog).** dbt parse can't help here, so the substitute is `test_brief_faithfulness.py`: every numeric token in the brief must be a substring of the serialized `Finding` numbers. That is the automated faithfulness gate that replaces `dry_run` for the prose layer.
- **Don't let the LLM drift into "diagnosing why."** CLAUDE.md and LEAN_SCOPE forbid causal claims - the prompt says the model narrates *where* and *how much*, never *why*. A brief sentence asserting a cause (a deploy, a price change) is a hallucination of fact and must be cut.

Reviewing generated code (specialize R1-R6)

- **R-faithfulness (the M7-specific gate, replaces R6 dry_run):** for the rendered brief, every figure must trace to a `Finding` field. Diff manually: take each dollar amount, percentage, p-value, and CI bound in the brief and find it verbatim in the input `Finding`. Zero orphan numbers. The automated `test_brief_faithfulness.py` must be green AND you read the brief once yourself.
- **R0 SQL count:** confirm `brief.py` issues **0** queries and contains **0** SQL strings - the brief step only calls the LLM and (optionally) `report.render_brief`. `grep` the diff for

`SELECT / build_query` inside `brief.py` : there should be none (all SQL lives upstream in the tested deterministic layer).

- **R1 cite:** the generated `brief.py` docstring cites AGENT_ARCHITECTURE 6.7 (Narrator faithfulness) and 4.3 (Finding shape); the runtime prompt is the M7 `render_brief` -only variant, not the full Narrator.
- **R2 import:** `python -c "import helios.brief"` clean; the function signature is `build_brief(finding: dict) → str`.
- **R3 ran the test:** Claude shows `pytest helios/tests/test_brief_faithfulness.py -q` passing, including the corrupted-brief case raising the error (if stretch built).
- **R4 canonical names:** every metric/dimension in the brief is in `semantic_layer.yaml` (grep). The `dominant` verdict in the prose matches `finding.dominant`.
- **R5 conventions + scope:** ONE LLM call per brief (two with self-critique) - not a loop, not an agent fleet. No FSM, no `save_diagnosis`, no memory. `revenue_at_risk` basis string is echoed, not recomputed. Verify wall-clock: the deterministic run + one brief completes in <5 min (L1 gate, AGENT_ARCHITECTURE sec 13).
- **Budget note:** run the brief sparingly during dev - test `brief.py` against the recorded fixture (no LLM), and reserve live LLM calls for final v1 validation and the M10 eval. Do not loop the brief while iterating on prompt wording; iterate against the fixture and the faithfulness test.

M8 - Memory Store [v2 - DEFERRED]

Deferred. Per LEAN_SCOPE (lines 40, 47, 90), memory / vector store / suppression / `action_tracking` is **cut** for the 2-week v1: it is over-engineering for a batch job on 3 months of frozen data, and the seasonality calendar only earns its keep once it *demonstrably reduces false positives*. The M7 brief does not need it. The notes below are how you *would* drive Claude Code to build it later - do not spend Pro budget or the 2 weeks on it now.

Context to load - `/clear`; rely on auto-loaded `CLAUDE.md`; attach

`@docs/HELIOS_PROJECT_BIBLE.md` **sec 22** (the 8 `helios_memory` tables, the seasonality/launch calendars, the `exp(-age/60d)` recall decay) and `@docs/MCP_ARCHITECTURE.md` sec 6.5 + sec on `save_diagnosis / recall_prior` (the only memory write path; idempotent on `finding_id`).

Files to provide - `@docs/HELIOS_PROJECT_BIBLE.md` sec 22 as the verbatim DDL source (the `CREATE TABLE IF NOT EXISTS` for `diagnosis_history`, `suppression_list`, `seasonality_calendar`, `launch_calendar`, `action_tracking`, `run_state`, `audit_log`, `glossary`); `@models/semantic/semantic_layer.yaml` for the canonical names the glossary maps to; the existing `@helios/mcp/report.py` to extend.

Prompts to use (sketch) - one focused session: "[ROLE] M8 (v2), file

`sql/helios_memory_ddl.sql` . [SOURCE OF TRUTH] copy the 8 `CREATE TABLE` statements verbatim from `@HELIOS_PROJECT_BIBLE.md` sec 22, including `PARTITION/CLUSTER`. [TASK] the DDL + a `seeds/memory/seasonality_calendar.csv` seeded with Black Friday 2020, the December peak, and the January trough. [CONSTRAINTS] use ONLY sec 22 column names; do not invent fields. [TEST-FIRST] write a `save_diagnosis` → `recall_prior` round-trip test against a fixture, then implement the `report-mcp` memory tools to pass it." A second prompt adds `save_diagnosis` / `recall_prior` to `report.py` keyed idempotently on `finding_id` .

Preventing hallucinations - D2/D3 with Bible sec 22 as the verbatim source; D5 forbids inventing memory columns; the round-trip test is the machine check (D6) - a saved finding must be retrievable by `(metric, segment)` and re-saving the same `finding_id` upserts (not duplicates). Seed the calendar from real dataset events only.

Reviewing generated code - R1 cites sec 22; R2 the DDL runs in BigQuery (`--location=US` to match the marts); R3 the round-trip test passes; R5 `report-mcp` is the ONLY memory writer (no agent holds a raw insert path), recall decays with `exp(-age/60)` and never hard-deletes (audit requirement).

Why deferred (be explicit): building 8 tables + a vector store + idempotent memory tools is several days and adds recurring LLM/embedding cost - it would blow the 2-week budget for a feature the frozen 3-month dataset cannot validate. Defer until v1 ships and you can show it cuts false positives.

M9 - Full 7-Agent Loop [v2 - DEFERRED]

Deferred. Per `LEAN_SCOPE` (lines 37, 90, 98) the 7-agent FSM is the single biggest cut: it is *apparatus disproportionate* to the value, and Claude Pro **cannot fund a fleet** of 7 agents x many BigQuery round-trips per run (`LEAN_SCOPE` line 21). The value - the decomposition and the grounding - is already delivered by M7's one grounded brief. The notes below are how you *would* drive Claude Code to build the full loop later; doing it inside the 2 weeks would exhaust both the calendar and the Pro budget.

Context to load - `/clear` ; auto-loaded `CLAUDE.md` ; attach `@docs/AGENT_ARCHITECTURE.md` **sec 4-10** (the `AgentSpec` , the tool-call wrapper, the `Finding` envelope, the FSM + sec 5.1 transition table, the per-agent specs 6.1-6.7, the G1-G5 double enforcement in sec 7, and sec 9 tunables) - plus `@docs/MCP_ARCHITECTURE.md` sec 10 (the authoritative per-agent allow-lists) and M8's memory tools (needed for the Critic's seasonality priors and the Narrator's `save_diagnosis`).

Files to provide - the M7 `helios/agents/framework.py` + `runner.py` to extend;
`@docs/AGENT_ARCHITECTURE.md` sec 6.x per agent; `@models/semantic/semantic_layer.yaml`
(the only metrics agents may name); recorded MCP fixtures for node unit tests (no live BigQuery).

Prompts to use (sketch) - this is where **subagents earn their keep** (the one place LEAN_SCOPE's "reserve subagents" rule relaxes): the five remaining agent files (`decompose.py` , `diagnose.py` , `critic.py` , `prescribe.py` , `orchestrator.py`) are largely independent, each a `*_SPEC` + system prompt + node logic copied from its sec 6.x. Drive one code-writing subagent per agent file in parallel, each handed only its own sec 6.x + its sec-10 allow-list, e.g.: "[ROLE] M9 (v2), file `helios/agents/critic.py` . [SOURCE OF TRUTH] copy the `CRITIC_SPEC` + 4-axis refutation battery from `@AGENT_ARCHITECTURE.md` sec 6.5; tools = exactly its `@MCP_ARCHITECTURE.md` sec 10 row. [TASK] the Critic node + its refutation logic. [CONSTRAINTS] the Critic's job is to REFUTE, default to skepticism, PASS only if all four refutations fail; no raw SQL (G1), no prose stats (G2). [TEST-FIRST] write `test_critic.py` with a mislabeled pure-mix-shift fixture asserting DROP/DOWNGRADE and a clean fixture asserting PASS, then implement." A final non-parallel prompt wires the full PLAN→MONITOR→DECOMPOSE→DIAGNOSE→[CRITIC]→PRESCRIBE→NARRATE FSM from sec 5.1. Use the **Critic as a self-check pass** - it is the production analog of M7's stretch self-critique, but with the full mix-shift / sample / seasonality / data-quality battery.

Preventing hallucinations - D2/D3 with sec 6.x as the verbatim source per agent; D5 forbids inventing tools or metrics; the structural defenses do the heavy lifting (sec 7): the framework wrapper `assert tool in agent.allowed_tools` makes out-of-scope tools impossible and `assert dry_run_seen(sql)` enforces G3 - so "the agent wrote its own SQL" is structurally impossible. The machine checks (D6): node unit tests against recorded fixtures, FSM transition tests, and the `audit_log` grounding scan (every `sql_text` originated from `semantic.build_query` ; every brief number maps to a tool-output hash).

Reviewing generated code - R1 each agent cites its sec 6.x; R2 nodes import and validate their Finding against `output_schema` ; R3 `test_critic.py` / `test_fsm_full.py` pass; R4 only `semantic_layer.yaml` names appear; R5 each agent's `allowed_tools` matches sec 10 exactly (e.g. Narrator cannot call `run_query` , Prescribe cannot touch `warehouse`) and Diagnose drills *rate* before *mix*; R6 leaf promotion requires `reconcile` $\leq 0.5\%$ drift.

Why deferred (be explicit): seven agents x Opus/Sonnet calls x multiple tool round-trips per run is an *API-fleet* cost profile that Claude Pro's usage limits cannot sustain - and the eval (M10) shows the lean one-brief path already beats the naive baseline. Build the full loop only as a funded v2, after M7/M10 prove the spine. The honest interview line is exactly this scoping decision (LEAN_SCOPE line 126): the full design was a 7-agent autonomous system; with 2 weeks and one dev you shipped the governed spine + one grounded LLM step + an honest eval, because the value was the decomposition and the grounding, not the fleet.